

Day 7: RAG, MCP and UiPath Integration for AI Automation

Enterprise AI Tool Integration for Insurance Automation | Professional Training Module

Program Context: AI Automation Engineering with Generative AI, Agentic AI, LangGraph and MCP for Insurance Domain UiPath Developers

Item	Details
Duration	Half Day - 4 Hours
Audience	UiPath Developers, RPA Developers, Automation Leads, Solution Architects, Insurance Automation Teams
Domain Focus	Insurance claims, policy lookup, claim validation, customer support, tool integration, auditability and governance
Primary Outcome	Build an AI Agent that invokes UiPath-supported insurance automation steps
Teaching Mode	Instructor-led, lab-driven, step-by-step hands-on implementation

1. Day 7 Learning Objective

The objective of Day 7 is to help participants understand how Retrieval-Augmented Generation, MCP-style tool integration, LangSmith observability and UiPath integration can work together to create enterprise-ready AI automation workflows for insurance use cases.

By the end of this session, participants should be able to design and explain an AI agent that retrieves insurance policy knowledge, invokes policy lookup and claim validation tools, and triggers UiPath-supported downstream automation using Python and REST APIs.

2. Recommended Learner Outcomes

- Explain RAG, embeddings and vector storage using insurance policy and claim knowledge examples.
- Build a basic RAG workflow using LangChain, embeddings and vector storage.
- Understand how LangGraph can orchestrate RAG, tool execution, validation and routing.
- Explain MCP fundamentals: server, client, tools and tool discovery.
- Expose policy lookup and claim validation as AI-callable tools.

- Use LangSmith conceptually for tracing, debugging and monitoring LangGraph or MCP-style workflows.
- Design UiPath integration patterns using Python services, REST APIs and robot invocation patterns.
- Apply security and governance controls for AI tool access in insurance workflows.

3. Day 7 Session Plan

Time	Topic	Practical Output
30 min	Quick introduction to RAG, embeddings and vector storage	Understand how policy documents become searchable knowledge
45 min	RAG implementation using LangChain and vector storage	Build policy FAQ retrieval workflow
35 min	LangGraph + RAG orchestration	Route question to retrieval, answer generation and fallback
40 min	MCP fundamentals and tool server concept	Expose policy lookup and claim validation as AI tools
25 min	LangSmith use case with LangGraph and MCP	Understand tracing, debugging and governance visibility
35 min	UiPath integration patterns using Python and REST APIs	Invoke Python AI service from UiPath and trigger robot concept
30 min	Security, governance and final assignment	Design governed AI agent invoking UiPath-supported insurance steps

4. Concept: What is RAG?

RAG stands for Retrieval-Augmented Generation. It improves LLM responses by retrieving relevant business knowledge before generating an answer. In insurance, RAG is useful because policy terms, exclusions, claim rules and regulatory language should come from trusted documents rather than model memory.

Traditional LLM Answer	RAG-Based Answer
May answer from general model knowledge.	Retrieves relevant policy clauses first, then answers.
May hallucinate coverage rules.	Can cite retrieved policy text or source document section.
Weak for internal company rules.	Strong for internal policy documents, SOPs and claim manuals.
Difficult to audit.	Better audit trail because retrieved context can be logged.

Insurance RAG Example

Customer question: Is room rent covered under my health insurance policy?

RAG workflow:

1. Convert policy documents into chunks.
2. Create embeddings for each chunk.
3. Store embeddings in a vector database.
4. Convert customer question into an embedding.
5. Retrieve the most relevant policy chunks.
6. Pass retrieved context to the LLM.
7. Generate a customer-friendly answer grounded in policy text.

5. Concept: Embeddings and Vector Storage

An embedding is a numerical representation of text meaning. Vector storage keeps these numerical representations so similar questions and policy clauses can be matched. This allows AI assistants to find relevant insurance knowledge even when the user asks the question in different words.

Term	Simple Explanation	Insurance Example
Document Chunk	Small piece of a document.	Room rent coverage clause from policy PDF.
Embedding	Numerical meaning of a text chunk.	Vector representing “hospitalization expenses covered up to INR 5,00,000”.
Vector Store	Database for embeddings.	ChromaDB or FAISS storing policy clauses.
Retriever	Search component that finds relevant chunks.	Finds room rent clause for customer question.
Grounded Answer	LLM response based on retrieved content.	Answer includes coverage limit and exclusions.

6. Lab 1: RAG Implementation using LangChain and Vector Storage

Business Scenario

A customer support team needs an AI assistant that can answer questions from health, motor and travel policy text. The assistant should retrieve relevant policy sections and answer only from the available context.

Step 1: Install Required Libraries

```
# Run only if libraries are missing
# pip install langchain langchain-openai chromadb python-dotenv openai
```

Step 2: Configure OpenRouter-Compatible LLM

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
OPENROUTER_BASE_URL = os.getenv("OPENROUTER_BASE_URL",
"https://openrouter.ai/api/v1")
OPENROUTER_MODEL = os.getenv("OPENROUTER_MODEL", "openai/gpt-4.1-
mini")

llm = ChatOpenAI(
    model=OPENROUTER_MODEL,
    api_key=OPENROUTER_API_KEY,
    base_url=OPENROUTER_BASE_URL,
    temperature=0.2
)
```

Step 3: Create Sample Insurance Policy Knowledge

```
policy_documents = [
```

"Health Secure Plus covers hospitalization expenses up to INR 5,00,000. Room rent is covered up to INR 5,000 per day.",

"Health Secure Plus excludes cosmetic treatment, experimental procedures and non-prescribed medicines.",

"Motor Comprehensive policy covers own damage, third-party liability, theft and natural calamity damage.",

"Motor claim documents include policy copy, driving license, RC copy, repair estimate, police report and photos.",

"Travel Shield covers baggage delay, trip delay and emergency medical expenses subject to policy limits.",

"Travel baggage delay claims require boarding pass, airline delay certificate, baggage delay report and claim form."

]

Step 4: Create Vector Store

```
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
```

```
embedding_model = OpenAIEmbeddings(
    model="text-embedding-3-small",
    api_key=OPENROUTER_API_KEY,
    base_url=OPENROUTER_BASE_URL
)
```

```
docs = [Document(page_content=text, metadata={"source": f"policy_{i+1}"}) for i,
text in enumerate(policy_documents)]
```

```
vectorstore = Chroma.from_documents(
    documents=docs,
    embedding=embedding_model,
    collection_name="insurance_policy_knowledge"
)
```

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

Trainer Note: If the selected provider does not support the embedding model through the same gateway, use an OpenAI embedding key or replace this section with local embeddings such as sentence-transformers. Keep the concept unchanged: text becomes vectors, vectors are searched, retrieved chunks are sent to the LLM.

Step 5: Build RAG Prompt and Chain

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

```
rag_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an insurance policy assistant. Answer only using the
provided context."),
    ("user", """
Question:
{question}
```

```
Retrieved Context:
{context}
```

Instructions:

- Answer in simple customer-friendly language.
- If answer is not available in context, say information is not available.
- Do not invent policy coverage.

```
""")
```

```
])
```

```
def format_docs(docs):
    return "
```

```
".join([doc.page_content for doc in docs])
```

```
def answer_policy_question(question):
```

```

retrieved_docs = retriever.invoke(question)
context = format_docs(retrieved_docs)
chain = rag_prompt | llm | StrOutputParser()
return chain.invoke({"question": question, "context": context})

```

```
print(answer_policy_question("Is room rent covered in the health policy?"))
```

Practice Questions

- What documents are required for a motor claim?
- Is cosmetic treatment covered?
- What documents are needed for baggage delay?
- Does the travel policy mention trip delay?
- What is the room rent limit in Health Secure Plus?

Mini Assignment 1

Add two more policy clauses for life insurance and property insurance. Rebuild the vector store and test at least three questions. Capture retrieved context and final answer.

7. Lab 2: RAG with LangGraph Step by Step

Business Scenario

The assistant should decide whether a question should go to RAG retrieval or fallback. If the retrieved context is insufficient, the workflow should route the question to human support.

Step 1: Define Graph State

```

from typing import TypedDict, List, Dict, Any
from langgraph.graph import StateGraph, START, END

class RAGState(TypedDict, total=False):
    question: str
    retrieved_context: str
    answer: str
    route: str

```

```
logs: List[str]
```

Step 2: Retrieval Node

```
def retrieve_policy_context(state: RAGState) -> RAGState:
    question = state["question"]
    docs = retriever.invoke(question)
    context = format_docs(docs)
    logs = state.get("logs", [])
    logs.append("Retriever node completed.")
    return {**state, "retrieved_context": context, "logs": logs}
```

Step 3: Answer Generation Node

```
def generate_rag_answer(state: RAGState) -> RAGState:
    chain = rag_prompt | llm | StrOutputParser()
    answer = chain.invoke({
        "question": state["question"],
        "context": state["retrieved_context"]
    })
    logs = state.get("logs", [])
    logs.append("Answer generation node completed.")
    return {**state, "answer": answer, "logs": logs}
```

Step 4: Routing Node

```
def evaluate_context_node(state: RAGState) -> RAGState:
    context = state.get("retrieved_context", "")
    route = "answer" if len(context.strip()) > 50 else "human_support"
    logs = state.get("logs", [])
    logs.append(f"Context evaluation completed. Route: {route}")
    return {**state, "route": route, "logs": logs}
```

```
def route_after_context(state: RAGState) -> str:
    return state.get("route", "human_support")
```

```
def human_support_node(state: RAGState) -> RAGState:
```



```

logs = state.get("logs", [])
logs.append("Human support node completed.")
return {
    **state,
    "answer": "Policy information is not available in the knowledge base. Please
route this query to human support.",
    "logs": logs
}

```

Step 5: Compile LangGraph RAG Workflow

```

rag_graph = StateGraph(RAGState)

rag_graph.add_node("retrieve", retrieve_policy_context)
rag_graph.add_node("evaluate", evaluate_context_node)
rag_graph.add_node("answer", generate_rag_answer)
rag_graph.add_node("human_support", human_support_node)

rag_graph.add_edge(START, "retrieve")
rag_graph.add_edge("retrieve", "evaluate")
rag_graph.add_conditional_edges(
    "evaluate",
    route_after_context,
    {
        "answer": "answer",
        "human_support": "human_support"
    }
)
rag_graph.add_edge("answer", END)
rag_graph.add_edge("human_support", END)

rag_app = rag_graph.compile()

```

```

result = rag_app.invoke({"question": "What documents are required for motor
claim?", "logs": []})
print(result["answer"])
print(result["logs"])

```

Mini Assignment 2

Add a confidence score node. If the answer contains “not available”, route to human support. Otherwise, return the answer to the customer support workflow.

8. Concept: MCP Fundamentals and Enterprise Relevance

MCP, or Model Context Protocol, is an open protocol for connecting AI applications to external data sources and tools. In an enterprise insurance setting, MCP-style integration can expose approved capabilities such as policy lookup, claim validation, document checklist generation, payment status lookup and UiPath robot triggering as tools.

MCP Components

Component	Meaning	Insurance Example
MCP Server	Exposes tools or data to AI clients.	Insurance tool server exposing policy lookup and claim validation.
MCP Client	Connects to server and discovers tools.	AI agent or desktop assistant discovering claim tools.
Tool	Callable business function.	validate_claim(policy_number, claim_amount).
Tool Discovery	Client learns available tools.	Agent sees policy_lookup, claim_validation, trigger_uipath_robot.
Governance Layer	Controls who can use which tool.	Only approved agents can trigger robot execution.

Why MCP Matters for Insurance Automation

- It helps standardize how AI agents discover and invoke business tools.
- It separates tool implementation from AI prompting.
- It improves reusability across agents, assistants and automation workflows.
- It supports governed AI access to enterprise functions.
- It provides a useful design pattern for agent-to-tool integration, even when implemented as a simple Python REST service in training.

9. Lab 3: Create a Simple MCP Tool Server Concept

Important Training Note

This lab uses a simplified MCP-style tool server concept. The purpose is to teach the architecture: tools are declared, discoverable and invocable. In production, teams should use the official MCP SDK and organizational security controls.

Step 1: Define Insurance Tools

```
policy_db = {
    "POL1001": {"customer": "Rajesh Sharma", "status": "Active", "coverage":
500000, "premium": "Paid"},
    "VEH2045": {"customer": "Meena Joshi", "status": "Active", "coverage": 300000,
"premium": "Paid"},
    "TRV3001": {"customer": "Amit Verma", "status": "Active", "coverage": 100000,
"premium": "Paid"}
}
```

```
def policy_lookup(policy_number: str):
    return policy_db.get(policy_number, {"error": "Policy not found"})
```

```
def claim_validation(policy_number: str, claim_amount: int):
    policy = policy_lookup(policy_number)
    if "error" in policy:
        return {"valid": False, "reason": "Policy not found"}
    if policy["status"] != "Active":
        return {"valid": False, "reason": "Policy inactive"}
    if policy["premium"] != "Paid":
        return {"valid": False, "reason": "Premium pending"}
    if claim_amount > policy["coverage"]:
        return {"valid": False, "reason": "Claim exceeds coverage"}
    return {"valid": True, "reason": "Claim is within active policy coverage"}
```

Step 2: Create Tool Registry

```
tool_registry = {
```

```

"policy_lookup": {
    "description": "Look up insurance policy status and coverage.",
    "function": policy_lookup,
    "input_schema": {"policy_number": "str"}
},
"claim_validation": {
    "description": "Validate claim amount against policy status and coverage.",
    "function": claim_validation,
    "input_schema": {"policy_number": "str", "claim_amount": "int"}
}
}

```

```

def list_tools():
    return {
        name: {
            "description": spec["description"],
            "input_schema": spec["input_schema"]
        }
        for name, spec in tool_registry.items()
    }

```

```
print(list_tools())
```

Step 3: Invoke Tool by Name

```

def invoke_tool(tool_name: str, arguments: dict):
    if tool_name not in tool_registry:
        return {"error": "Tool not found"}
    tool_function = tool_registry[tool_name]["function"]
    return tool_function(**arguments)

print(invoke_tool("policy_lookup", {"policy_number": "POL1001"}))
print(invoke_tool("claim_validation", {"policy_number": "POL1001", "claim_amount":
85000}))

```

Mini Assignment 3

Add a new tool named `document_checklist`. It should accept `claim_type` and return the required documents for Health, Motor and Travel claim types. Add it to the registry and test `list_tools()` and `invoke_tool()`.

10. Lab 4: Expose Policy Lookup and Claim Validation as FastAPI Tools

Business Scenario

UiPath or an AI agent needs to call policy lookup and claim validation over HTTP. We will expose the tools as FastAPI endpoints.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Insurance AI Tool Server")

class PolicyRequest(BaseModel):
    policy_number: str

class ClaimValidationRequest(BaseModel):
    policy_number: str
    claim_amount: int

@app.get("/tools")
def tools():
    return list_tools()

@app.post("/tools/policy_lookup")
def api_policy_lookup(request: PolicyRequest):
    return policy_lookup(request.policy_number)

@app.post("/tools/claim_validation")
def api_claim_validation(request: ClaimValidationRequest):
```

```
return claim_validation(request.policy_number, request.claim_amount)
```

Save as tool_server.py and run:

uvicorn tool_server:app --reload

Open: http://127.0.0.1:8000/docs

Mini Assignment 4

Create a third endpoint /tools/document_checklist. Test the endpoint from Swagger UI and from Python requests.

11. Lab 5: AI Agent that Uses Tools for Insurance Automation

Business Scenario

The AI agent receives a claim request, decides which tool to call, validates policy and claim amount, then prepares an action for UiPath.

```
def insurance_tool_agent(claim):
    policy_result = invoke_tool("policy_lookup", {"policy_number":
claim["policy_number"]})
    validation_result = invoke_tool("claim_validation", {
        "policy_number": claim["policy_number"],
        "claim_amount": claim["claim_amount"]
    })
```

```
prompt = f"""
```

You are an insurance automation agent.

Create a concise automation decision for UiPath.

Claim:

```
{claim}
```

Policy Result:

```
{policy_result}
```

Validation Result:

{validation_result}

Return:

- Decision
- Reason
- Recommended UiPath Action
- Customer Communication Summary

"""

```
response = llm.invoke(prompt)
```

```
return response.content
```

```
claim = {"policy_number": "POL1001", "claim_amount": 85000, "claim_type":  
"Health", "customer": "Rajesh Sharma"}
```

```
print(insurance_tool_agent(claim))
```

Mini Assignment 5

Modify the agent so it returns JSON output for UiPath with fields: claim_id, policy_number, claim_valid, reason, uipath_action, human_review_required.

12. Quick Introduction to LangSmith

LangSmith is used for tracing, debugging, evaluating and monitoring LLM and agent applications. For a LangGraph or MCP-style insurance workflow, LangSmith can help track each node execution, tool call, retrieved context, latency, token usage and failure points.

LangSmith Use Cases in This Program

Use Case	Why It Matters
Trace LangGraph claim routing	Understand why a claim moved to approval, rework or human review.
Monitor RAG retrieval quality	Check which policy clauses were retrieved for a customer question.
Debug MCP tool calls	Identify which tool was called and with what arguments.
Evaluate prompt changes	Compare claim summary quality before and after prompt improvement.

Audit AI decisions

Support responsible AI governance in insurance workflows.

13. Lab 6: Use Case of LangSmith with LangGraph and MCP

Step 1: Configure LangSmith Environment

```
# Add these to .env if LangSmith access is available
# LANGSMITH_TRACING=true
# LANGSMITH_API_KEY=your_langsmith_key
# LANGSMITH_PROJECT=insurance-rag-mcp-uipath-day7
```

Step 2: Trace LangGraph RAG Workflow

```
# When LangSmith environment variables are configured,
# LangChain and LangGraph calls can be traced automatically.
```

```
result = rag_app.invoke({
    "question": "What documents are required for a motor claim?",
    "logs": []
})
```

```
print(result["answer"])
print(result["logs"])
```

Step 3: Trace Tool Invocation

```
claim = {"policy_number": "POL1001", "claim_amount": 85000, "claim_type":
"Health"}
```

```
tool_decision = insurance_tool_agent(claim)
print(tool_decision)
```

```
# In LangSmith, review:
# - Prompt input
# - Retrieved context
```


- # - Tool calls
- # - LLM output
- # - Latency and errors

Mini Assignment 6

Run three different claims through the tool agent and compare traces. Identify which claim required human review and why.

14. UiPath Integration Patterns with Python and REST APIs

UiPath can integrate with AI automation services through several patterns. For enterprise insurance workflows, REST API integration is usually the cleanest because it separates UiPath robot execution from AI backend logic.

Pattern	Description	Insurance Example
Invoke Python Script	UiPath calls a local Python script.	Run claim validation script on extracted Excel data.
HTTP Request to FastAPI	UiPath sends JSON to Python AI service.	Call /claim_validation after document extraction.
Queue-Based Integration	UiPath queues claim data; Python worker processes claims.	Batch claims routed to RAG and LangGraph backend.
Robot Trigger from AI Workflow	AI service calls UiPath Orchestrator API or a robot trigger endpoint.	After policy validation, trigger robot for email, CRM update or claim system entry.
Human-in-the-Loop	AI routes high-risk claims to action center or approval task.	Fraud risk claim waits for claims manager approval.

15. Lab 7: Invoke Python AI Service from UiPath

Python AI Service Endpoint

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Insurance Claim AI Service")

class ClaimRequest(BaseModel):

```

```

claim_id: str
policy_number: str
claim_amount: int
claim_type: str

```

```

@app.post("/ai/claim-decision")
def claim_decision(request: ClaimRequest):
    validation = claim_validation(request.policy_number, request.claim_amount)
    return {
        "claim_id": request.claim_id,
        "policy_number": request.policy_number,
        "claim_valid": validation["valid"],
        "reason": validation["reason"],
        "recommended_uipath_action": "Proceed to downstream automation" if
validation["valid"] else "Route to manual review"
    }

```

```

# Run:
# uvicorn claim_ai_service:app --reload

```

UiPath Workflow Steps

8. Use Build Data Table or form input to prepare claim fields.
9. Use HTTP Request activity.
10. Set method to POST.
11. Set endpoint URL to `http://127.0.0.1:8000/ai/claim-decision`.
12. Pass JSON body with `claim_id`, `policy_number`, `claim_amount` and `claim_type`.
13. Deserialize JSON response.
14. Use If activity to route valid claims to downstream automation and invalid claims to manual review.

Sample JSON Request

```

{
  "claim_id": "CLM9001",
  "policy_number": "POL1001",
  "claim_amount": 85000,

```

```
"claim_type": "Health"
}
```

Mini Assignment 7

Create two test requests: one valid claim and one claim exceeding coverage. In UiPath, route the valid claim to “Proceed” and invalid claim to “Manual Review”.

16. Lab 8: Trigger UiPath Robot from AI-Assisted Workflow

Conceptual Architecture

The AI workflow does not directly perform all enterprise actions. It makes a governed decision and triggers an approved UiPath process for downstream execution such as CRM update, claim system entry, email generation or audit report creation.

Training Simulation

```
def trigger_uipath_robot(process_name: str, payload: dict):
    # Training simulation of UiPath trigger
    return {
        "triggered": True,
        "process_name": process_name,
        "payload": payload,
        "message": "UiPath robot trigger simulated successfully."
    }

def ai_to_uipath_orchestration(claim):
    validation = claim_validation(claim["policy_number"], claim["claim_amount"])
    if validation["valid"]:
        return trigger_uipath_robot("Insurance_Claim_Downstream_Automation",
claim)
    return {
        "triggered": False,
        "reason": validation["reason"],
        "next_action": "Manual review required"
    }
```

```
claim = {"claim_id": "CLM9001", "policy_number": "POL1001", "claim_amount":
85000, "claim_type": "Health"}
print(ai_to_uipath_orchestration(claim))
```

Mini Assignment 8

Modify the orchestration function to trigger different robot names:

Medical_Claim_Robot, Motor_Claim_Robot and Travel_Claim_Robot based on claim_type.

17. Security and Governance Considerations for AI Tool Access

Insurance workflows handle sensitive customer, policy and claim data. AI tool access must be governed, auditable and restricted. Agents should not have unlimited access to systems or ability to trigger robots without controls.

Risk	Control
Prompt injection through policy documents	Sanitize retrieved content and separate system instructions from retrieved text.
Unauthorized tool execution	Use allowlisted tools and role-based access control.
Sensitive customer data leakage	Mask or minimize PII in prompts and logs.
Wrong robot triggered by AI	Require deterministic validation and approval for high-impact actions.
Untraceable AI decisions	Log prompts, retrieved context, tool calls, user, timestamp and decision path.
Over-automation of claim approval	Use human review for high-value, high-risk or exception claims.

Governance Checklist

- Maintain an approved tool registry.
- Use role-based access for tool invocation.
- Require human approval for high-risk actions.
- Log every tool call and decision.
- Validate JSON before sending to UiPath.
- Separate development, test and production tools.
- Avoid storing raw PII in traces unless approved by policy.

- Create fallback routes for model, retrieval or tool failures.

18. Extra Practice Examples

Example 1: Health Policy Coverage Assistant

Build a RAG question-answering flow that answers health policy questions and routes unknown questions to human support.

Example 2: Motor Claim Document Assistant

Use MCP-style document_checklist tool to return motor claim documents and format the answer for a customer email.

Example 3: Travel Claim Validation Tool

Expose travel claim validation as a tool. Validate claim amount and required documents before triggering UiPath.

Example 4: LangSmith Debugging Exercise

Run one successful and one failed claim validation. Compare traces, inputs, outputs and latency in LangSmith.

Example 5: UiPath Robot Routing

Trigger different simulated UiPath processes based on claim_type and claim validation result.

19. Final Assignment: Build an AI Agent that Invokes UiPath-Supported Insurance Automation Steps

Assignment Objective

Participants must build an AI-powered insurance automation agent that retrieves policy knowledge, invokes tools, validates claims and triggers UiPath-supported downstream automation steps in a governed manner.

Required Components

Component	Required Output
RAG Knowledge Base	Policy FAQ using vector storage and retrieval.
LangGraph Workflow	Question routing, retrieval, answer generation and fallback path.
MCP-Style Tool Registry	Tool discovery and invocation for policy lookup and claim validation.

Tool Server Concept	FastAPI endpoints for policy lookup and claim validation.
LangSmith Observability	Conceptual or actual trace of retrieval, tools and graph execution.
UiPath Integration	HTTP Request pattern or simulated robot trigger.
Governance Controls	Tool allowlist, JSON validation, human review and audit log.

Final Workflow

15. Customer or UiPath submits a claim question or claim record.
16. RAG retrieves relevant policy context.
17. LangGraph decides whether answer, tool call or human support is needed.
18. MCP-style tool registry exposes policy lookup and claim validation.
19. AI agent invokes approved tool.
20. Python AI service returns structured JSON.
21. UiPath consumes JSON through HTTP Request activity.
22. UiPath robot performs downstream automation.
23. LangSmith or logs capture trace and audit details.

Submission Checklist

Requirement	Completed
RAG policy knowledge base created	☐
Vector store and retriever tested	☐
LangGraph RAG workflow created	☐
Policy lookup tool exposed	☐
Claim validation tool exposed	☐
FastAPI tool endpoint tested	☐
UiPath HTTP integration design completed	☐
Robot trigger simulation completed	☐
Security and governance checklist added	☐
Final demo prepared	☐

20. Review Questions for Discussion

24. Why is RAG safer than asking an LLM to answer policy questions from memory?
25. What is the role of embeddings in policy search?

26. When should an insurance AI assistant route a question to human support?
27. How does MCP-style tool discovery help enterprise AI integration?
28. Why should UiPath robot triggering be governed?
29. What should be traced in LangSmith for a LangGraph + MCP workflow?
30. What risks arise when AI agents can invoke tools?
31. How can JSON validation protect UiPath workflows?
32. Which insurance actions should always require human approval?
33. How can this Day 7 workflow support the final capstone?

21. Day 7 Summary

- RAG improves insurance answers by grounding them in policy documents.
- Embeddings and vector storage make policy knowledge searchable by meaning.
- LangGraph can orchestrate retrieval, tool usage, routing and fallback paths.
- MCP-style architecture separates AI reasoning from approved business tools.
- LangSmith supports tracing, debugging and governance visibility for agent workflows.
- UiPath can consume Python AI services using REST APIs and trigger downstream robots.
- Governance is essential when AI agents access customer data or enterprise tools.
- The final Day 7 outcome is an AI agent that invokes UiPath-supported insurance automation steps.

22. Reference Notes

This module aligns with the program objective of transforming UiPath developers into AI Automation Engineers using Python, Generative AI, LangChain, LangGraph, MCP, APIs and UiPath integration for insurance automation. It builds directly toward the Day 8 capstone, where Python data processing, Generative AI, LangGraph, MCP-style tools and UiPath workflow integration are combined into an end-to-end insurance claims automation platform.

Technology reference basis: LangChain documentation for vector store and integration concepts; LangGraph documentation for stateful orchestration, durable execution and human-in-the-loop capabilities; Model Context Protocol documentation for tool/server/client integration concepts; LangSmith documentation for tracing LangGraph applications.